

EKS Hybrid Node Air-gapped 로 설치하기

EKS Hybrid Node 설치 항목

- Nodeadm 설치 및 복사
- EKS Hybrid 설치 파일 목록(manifest.yaml)에 포함된 설치 파일 목록 추출
- shell script를 통해 S3 등에 저장
- Hybrid Node 등록하기

1. Nodeadm 설치

nodeadm은 EKS Hybrid Node 설치/구성을 위한 CLI tool 이며 manifest.yaml 을 참고하여 바이너리, 이미지 다운로드 해당 바이너리를 Private S3 Bucket에 저장 후 Hybrid Node에서 활용

다운로드 경로 : <https://hybrid-assets.eks.amazonaws.com/releases/latest/bin/linux/amd64/nodeadm>

참고 링크 : <https://docs.aws.amazon.com/eks/latest/userguide/hybrid-nodes-nodeadm.html>

```
curl -OL 'https://hybrid-assets.eks.amazonaws.com/releases/latest/bin/linux/amd64/nodeadm'
aws s3 cp nodeadm s3://ekshybrid-lhyehun/nodeadm
```

e.g) S3 URL : <https://ekshybrid-lhyehun.s3.ap-northeast-2.amazonaws.com/nodeadm>

2. EKS Hybrid 설치 파일 목록

인터넷(Bastion) 가능 환경에서 아래 Script 실행하여 S3버킷(VPC Endpoint 구성)에 업로드

=== 업로드된 파일 ===

INFO: 바이너리: 15개 → /binaries 하위 폴더에 저장

INFO: 체크섬: 12개 → /checksums 하위 폴더에 저장

INFO: 컨테이너 이미지: 5개 → ECR 이미지 리스트 container_images.txt 확인 가능

참고

- 버전별 설치 경로는 <https://hybrid-assets.eks.amazonaws.com/manifest.yaml> 에 포함되어 있으며 해당 도메인을 proxy에 등록하여 내부에서 사용 하면 됨

- nodeadm 자체에서 manifest.yaml 을 별도로 load하는 기능은 현재 미지원

```
nodeadm install 1.33 --credential-provider ssm
```

NODEADM으로 설치된 바이너리 및 설치 경로

아티팩트	경로
IAM Roles Anywhere CLI	/usr/local/bin/aws_signing_helper
Kubelet 바이너리	/usr/bin/kubelet
Kubectl 바이너리	usr/local/bin/kubectl
ECR 자격 증명 공급자	/etc/eks/image-credential-provider/ecr-credential-provider
AWS IAM 인증자	/usr/local/bin/aws-iam-authenticator
SSM 설정 CLI	/opt/ssm/ssm-setup-cli
SSM 에이전트	Ubuntu - /snap/amazon-ssm-agent/current/amazon-ssm-agent RHEL 및 AL2023 - /usr/bin/amazon-ssm-agent
Containerd	Ubuntu 및 AL2023 - /usr/bin/containerd RHEL - /bin/containerd
Iptables	Ubuntu 및 AL2023 - /usr/sbin/iptables RHEL - /sbin/iptables
CNI 플러그인	/opt/cni/bin
설치된 아티팩트 트래커	/opt/nodeadm/tracker

MANIFEST.YAML 파일 다운로드 및 S3로 복제

```
# 인터넷 연결된 환경에서 매니페스트 다운로드 후 S3 업로드
curl -L https://hybrid-assets.eks.amazonaws.com/manifest.yaml -o manifest.yaml
aws s3 cp manifest.yaml s3://ekshybrid-lhyehun/manifest.yaml
```

manifest.yaml에는 아래와 같이 실제 설치 경로가 포함되어 있으며 해당 리스트를 참고하여 설치 진행함

- 단, 아직까지는 EKS Anywhere 와 같이 `—bundles-override` 이 지원되지 않음

MANIFEST.YAML 내의 ARTIFACTS

```
- latest_patch_version: "3"
  major_minor_version: "1.33"
  patch_releases:
  - artifacts:
    - arch: amd64
      checksum_uri: https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/darwin-aws-iam-authenticator
      name: aws-iam-authenticator
      os: darwin
      uri: https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/darwin-aws-iam-authenticator
```

- arch: amd64
checksum_uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/darwin/kubectl>
name: kubectl
os: darwin
uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/darwin/kubectl>
- arch: amd64
checksum_uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/aws-iam-authenticator>
name: aws-iam-authenticator
os: linux
uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/aws-iam-authenticator>
- arch: amd64
checksum_uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/cni>
name: cni
os: linux
uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/cni>
- arch: amd64
checksum_uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/cni-plugins>
name: cni-plugins
os: linux
uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/cni-plugins>
- arch: amd64
checksum_uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/ecr-credential-provider>
name: ecr-credential-provider
os: linux
uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/ecr-credential-provider>
- arch: amd64
checksum_uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/kube-proxy>
name: kube-proxy
os: linux
uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/kube-proxy>
- arch: amd64
checksum_uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/kubectl>
name: kubectl
os: linux
uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/kubectl>
- arch: amd64
checksum_uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/kubelet>
name: kubelet
os: linux
uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/kubelet>
- arch: arm64
checksum_uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/aws-iam-authenticator>
name: aws-iam-authenticator
os: linux
uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/aws-iam-authenticator>
- arch: arm64
checksum_uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/cni>
name: cni
os: linux
uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/cni>
- arch: arm64
checksum_uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/cni-plugins>
name: cni-plugins
os: linux
uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/cni-plugins>

- arch: arm64
checksum_uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/arm64/ecr-credential-provider>
name: ecr-credential-provider
os: linux
uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/arm64/ecr-credential-provider>
- arch: arm64
checksum_uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/arm64/kube-proxy>
name: kube-proxy
os: linux
uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/arm64/kube-proxy>
- arch: arm64
checksum_uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/arm64/kubectld>
name: kubectld
os: linux
uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/arm64/kubectld>
- arch: arm64
checksum_uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/arm64/kubelet>
name: kubelet
os: linux
uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/linux/arm64/kubelet>
- arch: amd64
checksum_uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/windows/amd64/aws-iam-authenticator>
name: aws-iam-authenticator
os: windows
uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/windows/amd64/aws-iam-authenticator>
- arch: amd64
checksum_uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/windows/amd64/ecr-credential-provider>
name: ecr-credential-provider
os: windows
uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/windows/amd64/ecr-credential-provider>
- arch: amd64
checksum_uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/windows/amd64/kube-proxy>
name: kube-proxy
os: windows
uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/windows/amd64/kube-proxy>
- arch: amd64
checksum_uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/windows/amd64/kubectld>
name: kubectld
os: windows
uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/windows/amd64/kubectld>
- arch: amd64
checksum_uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/windows/amd64/kubelet>
name: kubelet
os: windows
uri: <https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01/bin/windows/amd64/kubelet>

BINARIES FOR EKS HYBRID NODE

binaries/

Objects | Properties

Objects (15)   Copy S3 URI  Copy URL  Download  Open  Delete

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your object [more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	aws_signing_helper	-	August 12, 2025, 17:03:59 (UTC+09:00)	12.5 MB	Standard
☐	aws-iam-authenticator	-	August 12, 2025, 17:03:59 (UTC+09:00)	82.4 MB	Standard
☐	aws-iam-authenticator.gz	gz	August 12, 2025, 17:03:59 (UTC+09:00)	19.5 MB	Standard
☐	cni-amd64-v0.6.0.tgz	tgz	August 12, 2025, 17:03:59 (UTC+09:00)	1.8 MB	Standard
☐	cni-plugins-linux-amd64-v1.6.1.tgz	tgz	August 12, 2025, 17:03:59 (UTC+09:00)	50.3 MB	Standard
☐	ecr-credential-provider	-	August 12, 2025, 17:03:59 (UTC+09:00)	19.9 MB	Standard
☐	ecr-credential-provider.gz	gz	August 12, 2025, 17:04:00 (UTC+09:00)	6.7 MB	Standard
☐	kube-proxy	-	August 12, 2025, 17:04:00 (UTC+09:00)	67.4 MB	Standard
☐	kube-proxy.gz	gz	August 12, 2025, 17:04:00 (UTC+09:00)	18.9 MB	Standard
☐	kubectl	-	August 12, 2025, 17:04:00 (UTC+09:00)	57.4 MB	Standard

CONTAINER IMAGE IN ECR

602401143452.dkr.ecr.ap-northeast-2.amazonaws.com/eks/kube-proxy:v1.33.3-minimal-eksbuild.1
 602401143452.dkr.ecr.ap-northeast-2.amazonaws.com/eks/pause:3.5
 602401143452.dkr.ecr.ap-northeast-2.amazonaws.com/amazon-k8s-cni:v1.18.5-eksbuild.1
 602401143452.dkr.ecr.ap-northeast-2.amazonaws.com/amazon-k8s-cni-init:v1.18.5-eksbuild.1
 602401143452.dkr.ecr.ap-northeast-2.amazonaws.com/eks/coredns:v1.11.3-eksbuild.1

3. 설치 스크립트

아래 shell을 인터넷 되는 환경에서 실행 진행 하게 되면 아래 S3버킷에 해당 K8S 버전의 바이너리를 저장하게 됨

```
./ekshybrid-downolad.sh <S3버킷명> [쿠버네티스 버전]
예시 : ./ekshybrid-downolad.sh ekshybrid-lhyehun 1.32
```

```
#!/bin/bash

# EKS Hybrid nodeadm 폐쇄망 설치 스크립트
# 사용법: ./nodeadm-offline-setup.sh <S3_BUCKET_NAME> [KUBERNETES_VERSION] [ARCHITECTURE]

set -e

# 기본값 설정
```

```

KUBERNETES_VERSION="${2:-1.33.3}"
ARCHITECTURE="${3:-amd64}"
REGION="ap-northeast-2"
MANIFEST_URL="https://hybrid-assets.eks.amazonaws.com/manifest.yaml"
WORK_DIR="/tmp/nodeadm-offline"
LOG_FILE="/tmp/nodeadm-offline-setup.log"

# 색상 정의
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
BLUE='\033[0;34m'
NC='\033[0m' # No Color

# 로깅 함수
log() {
    echo -e "${GREEN}[$(date '+%Y-%m-%d %H:%M:%S')] $1${NC}" | tee -a "$LOG_FILE"
}

warn() {
    echo -e "${YELLOW}[$(date '+%Y-%m-%d %H:%M:%S')] WARNING: $1${NC}" | tee -a "$LOG_FILE"
}

error() {
    echo -e "${RED}[$(date '+%Y-%m-%d %H:%M:%S')] ERROR: $1${NC}" | tee -a "$LOG_FILE"
    exit 1
}

info() {
    echo -e "${BLUE}[$(date '+%Y-%m-%d %H:%M:%S')] INFO: $1${NC}" | tee -a "$LOG_FILE"
}

# 사용법 출력
usage() {
    echo "사용법: $0 <S3_BUCKET_NAME> [KUBERNETES_VERSION] [ARCHITECTURE]"
    echo ""
    echo "매개변수:"
    echo "  S3_BUCKET_NAME      : 바이너리와 이미지를 저장할 S3 버킷명 (필수)"
    echo "  KUBERNETES_VERSION  : Kubernetes 버전 (기본값: 1.33.3)"
    echo "  ARCHITECTURE        : 아키텍처 (기본값: amd64, 옵션: arm64)"
    echo ""
    echo "예시:"
    echo "  $0 my-nodeadm-bucket"
    echo "  $0 my-nodeadm-bucket 1.33.3 amd64"
    exit 1
}

# 매개변수 검증
if [ $# -lt 1 ]; then
    error "S3 버킷명이 필요합니다."
    usage
fi

S3_BUCKET="$1"

```

```

# 필수 도구 확인
check_prerequisites() {
    log "필수 도구 확인 중..."

    local missing_tools=()

    if ! command -v aws &> /dev/null; then
        missing_tools+=("aws-cli")
    fi

    if ! command -v curl &> /dev/null; then
        missing_tools+=("curl")
    fi

    if ! command -v python3 &> /dev/null; then
        missing_tools+=("python3")
    fi

    # PyYAML 확인
    if ! python3 -c "import yaml" &> /dev/null; then
        warn "PyYAML이 설치되지 않았습니다. 설치를 시도합니다..."
        if command -v pip3 &> /dev/null; then
            pip3 install PyYAML
        elif command -v yum &> /dev/null; then
            sudo yum install -y python3-pip && pip3 install PyYAML
        else
            missing_tools+=("PyYAML")
        fi
    fi

    if [ ${#missing_tools[@]} -ne 0 ]; then
        error "다음 도구들이 필요합니다: ${missing_tools[*]}"
    fi

    log "필수 도구 확인 완료"
}

# 작업 디렉터리 생성
setup_work_directory() {
    log "작업 디렉터리 설정 중..."
    rm -rf "$WORK_DIR"
    mkdir -p "$WORK_DIR"/{binaries,checksums,images}
    cd "$WORK_DIR"
    log "작업 디렉터리: $WORK_DIR"
}

# manifest.yaml 다운로드 및 파싱
download_manifest() {
    log "manifest.yaml 다운로드 중..."
    curl -s "$MANIFEST_URL" -o manifest.yaml

    if [ ! -f manifest.yaml ]; then
        error "manifest.yaml 다운로드 실패"
    fi
}

```

```

    log "manifest.yaml 다운로드 완료"
}

# Python 파서 스크립트 생성
create_parser_script() {
    log "Python 파서 스크립트 생성 중..."

    cat > parse_manifest.py << 'EOF'
#!/usr/bin/env python3

import yaml
import json
import sys
import os

def extract_binary_urls(manifest_file, kubernetes_version, architecture='amd64', os_type='linux'):
    try:
        with open(manifest_file, 'r') as f:
            data = yaml.safe_load(f)
    except Exception as e:
        print(f"Error reading manifest file: {e}", file=sys.stderr)
        return None, None

    major_minor = '.'.join(kubernetes_version.split('.')[0:2])

    binary_urls = []
    checksum_urls = []

    # supported_eks_releases에서 해당 버전 찾기
    for release in data.get('supported_eks_releases', []):
        if release.get('major_minor_version') == major_minor:
            for patch_release in release.get('patch_releases', []):
                if patch_release.get('version') == kubernetes_version:
                    for artifact in patch_release.get('artifacts', []):
                        if (artifact.get('os') == os_type and
                            artifact.get('arch') == architecture):
                            binary_urls.append(artifact.get('uri'))
                            checksum_urls.append(artifact.get('checksum_uri'))
                    break
            break

    return binary_urls, checksum_urls

def extract_additional_urls(manifest_file, region='ap-northeast-2', architecture='amd64', os_type='linux'):
    try:
        with open(manifest_file, 'r') as f:
            data = yaml.safe_load(f)
    except Exception as e:
        print(f"Error reading manifest file: {e}", file=sys.stderr)
        return {}

    additional_urls = {}

    # IAM Roles Anywhere releases
    for release in data.get('iam_roles_anywhere_releases', []):

```



```

        for artifact in release.get('artifacts', []):
            if artifact.get('os') == 'linux' and artifact.get('arch') == architecture:
                additional_urls['aws_signing_helper'] = {
                    'uri': artifact.get('uri'),
                    'checksum_uri': artifact.get('checksum_uri')
                }
                break

    # Region specific ECR account ID
    region_config = data.get('region_config', {}).get(region, {})
    ecr_account_id = region_config.get('ecr_account_id', '602401143452')

    additional_urls['ecr_account_id'] = ecr_account_id
    additional_urls['nodeadm'] = f"https://hybrid-assets.eks.amazonaws.com/releases/latest/nodeadm"
    additional_urls['ssm_setup_cli'] = f"https://amazon-ssm-us-west-2.s3.us-west-2.amazonaws.com/latest/setup-cli"

    return additional_urls

def generate_container_images(ecr_account_id, region, kubernetes_version):
    ecr_registry = f"{ecr_account_id}.dkr.ecr.{region}.amazonaws.com"

    images = [
        f"{ecr_registry}/eks/kube-proxy:v{kubernetes_version}-minimal-eksbuild.1",
        f"{ecr_registry}/eks/pause:3.5",
        f"{ecr_registry}/amazon-k8s-cni:v1.18.5-eksbuild.1",
        f"{ecr_registry}/amazon-k8s-cni-init:v1.18.5-eksbuild.1",
        f"{ecr_registry}/eks/coredns:v1.11.3-eksbuild.1"
    ]

    return images

if __name__ == '__main__':
    if len(sys.argv) < 4:
        print("Usage: python3 parse_manifest.py <manifest_file> <version> <arch> [region]")
        sys.exit(1)

    manifest_file = sys.argv[1]
    version = sys.argv[2]
    arch = sys.argv[3]
    region = sys.argv[4] if len(sys.argv) > 4 else 'ap-northeast-2'

    binary_urls, checksum_urls = extract_binary_urls(manifest_file, version, arch)

    if not binary_urls:
        print(f"No binaries found for version {version} and architecture {arch}", file=sys.stderr)
        sys.exit(1)

    additional_urls = extract_additional_urls(manifest_file, region, arch)
    ecr_account_id = additional_urls.get('ecr_account_id', '602401143452')
    container_images = generate_container_images(ecr_account_id, region, version)

    # 파일로 저장
    with open('binary_urls.txt', 'w') as f:
        for url in binary_urls:
            if url:

```

```

        f.write(url + '\n')

    with open('checksum_urls.txt', 'w') as f:
        for url in checksum_urls:
            if url:
                f.write(url + '\n')

    with open('container_images.txt', 'w') as f:
        for image in container_images:
            f.write(image + '\n')

    with open('additional_urls.json', 'w') as f:
        json.dump(additional_urls, f, indent=2)

    print(f"Extracted {len(binary_urls)} binaries, {len(checksum_urls)} checksums, {len(container_images)} container images, {len(additional_urls)} additional URLs")
EOF

    chmod +x parse_manifest.py
    log "Python 파서 스크립트 생성 완료"
}

# Kubernetes 버전별 바이너리 URL 추출
extract_binary_urls() {
    log "Kubernetes $KUBERNETES_VERSION ($ARCHITECTURE) 바이너리 URL 추출 중..."

    python3 parse_manifest.py manifest.yaml "$KUBERNETES_VERSION" "$ARCHITECTURE" "$RELEASE"

    if [ ! -s binary_urls.txt ]; then
        error "지정된 Kubernetes 버전 ($KUBERNETES_VERSION) 또는 아키텍처 ($ARCHITECTURE)에 대한 바이너리 URL이 없습니다."
    fi

    log "바이너리 URL 추출 완료 ($(wc -l < binary_urls.txt)개 파일)"
}

# 바이너리 다운로드
download_binaries() {
    log "바이너리 다운로드 중..."

    local count=0
    local total=$(wc -l < binary_urls.txt)

    while IFS= read -r url; do
        if [ -n "$url" ]; then
            count=$((count + 1))
            filename=$(basename "$url")
            info "[${count}/${total}] $filename 다운로드 중..."

            if curl -L -o "binaries/$filename" "$url"; then
                log "✓ $filename 다운로드 완료"
            else
                warn "✗ $filename 다운로드 실패"
            fi
        fi
    done < binary_urls.txt
}

```

```

    log "바이너리 다운로드 완료"
}

# 체크섬 다운로드
download_checksums() {
    log "체크섬 파일 다운로드 중..."

    local count=0
    local total=$(wc -l < checksum_urls.txt)

    while IFS= read -r url; do
        if [ -n "$url" ]; then
            count=$((count + 1))
            filename=$(basename "$url")
            info "[$count/$total] $filename 다운로드 중..."

            if curl -L -o "checksums/$filename" "$url"; then
                log "✓ $filename 다운로드 완료"
            else
                warn "✗ $filename 다운로드 실패"
            fi
        fi
    done < checksum_urls.txt

    log "체크섬 다운로드 완료"
}

# 추가 필수 바이너리 다운로드
download_additional_binaries() {
    log "추가 필수 바이너리 다운로드 중..."

    # additional_urls.json에서 URL 읽기
    if [ -f additional_urls.json ]; then
        # nodeadm 바이너리
        nodeadm_url=$(python3 -c "import json; data=json.load(open('additional_urls.json')); print(data['nodeadm'])")
        if [ -n "$nodeadm_url" ]; then
            info "nodeadm 바이너리 다운로드 중..."
            curl -L -o "binaries/nodeadm" "$nodeadm_url"
            chmod +x "binaries/nodeadm"
        fi

        # SSM agent installer
        ssm_url=$(python3 -c "import json; data=json.load(open('additional_urls.json')); print(data['ssm'])")
        if [ -n "$ssm_url" ]; then
            info "SSM agent installer 다운로드 중..."
            curl -L -o "binaries/ssm-setup-cli" "$ssm_url"
            chmod +x "binaries/ssm-setup-cli"
        fi

        # AWS signing helper
        signing_helper=$(python3 -c "import json; data=json.load(open('additional_urls.json')); print(data['aws_signing_helper'])")
        if [ -n "$signing_helper" ]; then
            info "AWS signing helper 다운로드 중..."
            curl -L -o "binaries/aws_signing_helper" "$signing_helper"
            chmod +x "binaries/aws_signing_helper"
        fi
    fi
}

```

```

        fi
    fi

    log "추가 바이너리 다운로드 완료"
}

# 체크섬 검증
verify_checksums() {
    log "체크섬 검증 중..."

    local verified=0
    local failed=0

    for checksum_file in checksums/*.sha256; do
        if [ -f "$checksum_file" ]; then
            checksum_name=$(basename "$checksum_file" .sha256)
            binary_file="binaries/$checksum_name"

            if [ -f "$binary_file" ]; then
                expected_checksum=$(cat "$checksum_file" | awk '{print $1}')
                actual_checksum=$(sha256sum "$binary_file" | awk '{print $1}')

                if [ "$expected_checksum" = "$actual_checksum" ]; then
                    info "✓ $checksum_name 체크섬 검증 성공"
                    verified=$((verified + 1))
                else
                    warn "✗ $checksum_name 체크섬 검증 실패"
                    failed=$((failed + 1))
                fi
            fi
        fi
    done

    log "체크섬 검증 완료: 성공 $verified개, 실패 $failed개"
}

# S3에 업로드
upload_to_s3() {
    log "S3 버킷 ($S3_BUCKET)에 파일 업로드 중..."

    # S3 버킷 존재 확인
    if ! aws s3 ls "s3://$S3_BUCKET" --region "$REGION" &>/dev/null; then
        info "S3 버킷 생성 중..."
        aws s3 mb "s3://$S3_BUCKET" --region "$REGION"
    fi

    # manifest.yaml 업로드
    aws s3 cp manifest.yaml "s3://$S3_BUCKET/manifest.yaml" --region "$REGION"

    # 바이너리 업로드
    info "바이너리 파일 업로드 중..."
    aws s3 sync binaries/ "s3://$S3_BUCKET/binaries/" --region "$REGION"

    # 체크섬 업로드
    info "체크섬 파일 업로드 중..."
}

```

```

aws s3 sync checksums/ "s3://$S3_BUCKET/checksums/" --region "$REGION"

# 이미지 목록 및 메타데이터 업로드
info "메타데이터 파일 업로드 중..."
aws s3 cp container_images.txt "s3://$S3_BUCKET/container_images.txt" --region "$REGION"
aws s3 cp additional_urls.json "s3://$S3_BUCKET/additional_urls.json" --region "$REGION"
aws s3 cp binary_urls.txt "s3://$S3_BUCKET/binary_urls.txt" --region "$REGION"
aws s3 cp checksum_urls.txt "s3://$S3_BUCKET/checksum_urls.txt" --region "$REGION"

log "S3 업로드 완료"
}

# 폐쇄망 설치 스크립트 생성
create_offline_install_script() {
    log "폐쇄망 설치 스크립트 생성 중..."

    cat > offline-install.sh << 'EOF'
#!/bin/bash

# EKS Hybrid nodeadm 폐쇄망 설치 스크립트
# 사용법: ./offline-install.sh <S3_BUCKET_NAME> <VPC_ENDPOINT_URL>

set -e

S3_BUCKET="$1"
VPC_ENDPOINT_URL="$2"
REGION="ap-northeast-2"

if [ $# -lt 2 ]; then
    echo "사용법: $0 <S3_BUCKET_NAME> <VPC_ENDPOINT_URL>"
    echo "예시: $0 my-nodeadm-bucket https://vpce-xxxxx-xxxxx.s3.ap-northeast-2.vpce.amazonaws.com"
    exit 1
fi

log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1"
}

# AWS CLI S3 엔드포인트 설정
export AWS_S3_ENDPOINT_URL="$VPC_ENDPOINT_URL"

# 작업 디렉터리 생성
WORK_DIR="/tmp/nodeadm-install"
mkdir -p "$WORK_DIR"
cd "$WORK_DIR"

log "S3에서 파일 다운로드 중..."

# 메타데이터 파일들 다운로드
aws s3 cp "s3://$S3_BUCKET/manifest.yaml" . --region "$REGION"
aws s3 cp "s3://$S3_BUCKET/container_images.txt" . --region "$REGION"
aws s3 cp "s3://$S3_BUCKET/additional_urls.json" . --region "$REGION"

# 바이너리 다운로드
mkdir -p binaries

```

```

aws s3 sync "s3://$S3_BUCKET/binaries/" binaries/ --region "$REGION"

# 체크섬 다운로드
mkdir -p checksums
aws s3 sync "s3://$S3_BUCKET/checksums/" checksums/ --region "$REGION"

# 체크섬 검증
log "체크섬 검증 중..."
for checksum_file in checksums/*.sha256; do
    if [ -f "$checksum_file" ]; then
        checksum_name=$(basename "$checksum_file" .sha256)
        binary_file="binaries/$checksum_name"

        if [ -f "$binary_file" ]; then
            expected_checksum=$(cat "$checksum_file" | awk '{print $1}')
            actual_checksum=$(sha256sum "$binary_file" | awk '{print $1}')

            if [ "$expected_checksum" = "$actual_checksum" ]; then
                log "✓ $checksum_name 체크섬 검증 성공"
            else
                log "✗ $checksum_name 체크섬 검증 실패"
                exit 1
            fi
        fi
    fi
done

# 바이너리 설치
log "바이너리 설치 중..."
sudo cp binaries/nodeadm /usr/local/bin/ 2>/dev/null || true
sudo cp binaries/kubectrl /usr/local/bin/ 2>/dev/null || true
sudo cp binaries/kubelet /usr/local/bin/ 2>/dev/null || true
sudo cp binaries/aws-iam-authenticator /usr/local/bin/ 2>/dev/null || true
sudo cp binaries/ecr-credential-provider /usr/local/bin/ 2>/dev/null || true
sudo cp binaries/kube-proxy /usr/local/bin/ 2>/dev/null || true

# 실행 권한 부여
sudo chmod +x /usr/local/bin/{nodeadm,kubectrl,kubelet,aws-iam-authenticator,ecr-creder

# CNI 플러그인 설치
log "CNI 플러그인 설치 중..."
sudo mkdir -p /opt/cni/bin
if [ -f binaries/cni-plugins-linux-*.tgz ]; then
    sudo tar -xzf binaries/cni-plugins-linux-*.tgz -C /opt/cni/bin/
fi
if [ -f binaries/cni-*-v0.6.0.tgz ]; then
    sudo tar -xzf binaries/cni-*-v0.6.0.tgz -C /opt/cni/bin/
fi

# containerd 설정 (필요한 경우)
if command -v containerd &> /dev/null; then
    log "containerd 설정 중..."
    sudo mkdir -p /etc/containerd
    if [ ! -f /etc/containerd/config.toml ]; then
        sudo containerd config default | sudo tee /etc/containerd/config.toml > /dev/r

```

```

    fi
fi

# 컨테이너 이미지 사전 로드 (Docker가 설치된 경우)
if command -v docker &> /dev/null && [ -f container_images.txt ]; then
    log "컨테이너 이미지 사전 로드 중..."
    while IFS= read -r image; do
        log "이미지 pull: $image"
        docker pull "$image" || log "이미지 pull 실패: $image"
    done < container_images.txt
fi

log "폐쇄망 설치 완료!"
log ""
log "다음 단계:"
log "1. nodeadm 버전 확인: nodeadm version"
log "2. EKS 컴포넌트 설치: sudo nodeadm install 1.33 --credential-provider ssm"
log "3. 클러스터 조인: sudo nodeadm init --cluster-name <CLUSTER_NAME>"

EOF

    chmod +x offline-install.sh

    # S3에 업로드
    aws s3 cp offline-install.sh "s3://$S3_BUCKET/offline-install.sh" --region "$REGION"

    log "폐쇄망 설치 스크립트 생성 및 업로드 완료"
}

# 요약 정보 출력
print_summary() {
    log "=== 설정 완료 요약 ==="
    info "S3 버킷: $S3_BUCKET"
    info "Kubernetes 버전: $KUBERNETES_VERSION"
    info "아키텍처: $ARCHITECTURE"
    info "리전: $REGION"
    info "작업 디렉터리: $WORK_DIR"
    echo ""
    log "=== 업로드된 파일 통계 ==="
    info "바이너리: $(ls binaries/ | wc -l)개"
    info "체크섬: $(ls checksums/ | wc -l)개"
    info "컨테이너 이미지: $(wc -l < container_images.txt)개"
    echo ""
    log "=== 폐쇄망에서 설치 방법 ==="
    info "1. VPC Endpoint를 통해 S3에 접근 가능한 환경에서 다음 명령 실행:"
    info "    aws s3 cp s3://$S3_BUCKET/offline-install.sh . --region $REGION --endpoint-url $(cat /dev/urandom | tr -dc 'a-z0-9' | fold -n 40 | xargs -n1 sha256sum | cut -d' ' -f1)"
    info "    chmod +x offline-install.sh"
    info "    sudo ./offline-install.sh $S3_BUCKET <VPC_ENDPOINT_URL>"
    echo ""
    info "2. VPC Endpoint URL 예시:"
    info "    https://vpce-xxxxx-xxxxx.s3.ap-northeast-2.vpce.amazonaws.com"
    echo ""
    log "=== 업로드된 주요 파일들 ==="
    aws s3 ls "s3://$S3_BUCKET/" --region "$REGION" | head -10
    echo "..."
}

```

```

    info "전체 파일 목록: aws s3 ls s3://$S3_BUCKET/ --recursive --region $REGION"
}

# 메인 실행 함수
main() {
    log "EKS Hybrid nodeadm 페쇄망 설치 준비 시작"
    log "S3 버킷: $S3_BUCKET, Kubernetes: $KUBERNETES_VERSION, 아키텍처: $ARCHITECTURE"

    check_prerequisites
    setup_work_directory
    download_manifest
    create_parser_script
    extract_binary_urls
    download_binaries
    download_checksums
    download_additional_binaries
    verify_checksums
    upload_to_s3
    create_offline_install_script
    print_summary

    log "모든 작업이 완료되었습니다!"
}

# 스크립트 실행
main "$@"

```

4. 노드 등록하기

```

## 다음 단계: eksworkshop 클러스터에 노드 등록

### 1. nodeadm 설정 파일 생성

cat > eksworkshop-nodeadm-config.yaml << 'EOF'
apiVersion: node.eks.aws/v1alpha1
kind: NodeConfig
spec:
  cluster:
    name: eksworkshop
    apiServerEndpoint: https://E335762DAB06FA9AE2A51A8EEE9CC73D.gr7.ap-northeast-2.eks
    certificateAuthority: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURSB0tLS0tCk1JSURCVENDQWUyZ0F3SUJE
    cidr: 172.20.0.0/16
  kubelet:
    config:
      clusterDNS:
        - 172.20.0.10
    flags:
      - --node-labels=node.kubernetes.io/instance-type=hybrid,topology.kubernetes.io/z
      - --register-with-taints=node.kubernetes.io/instance-type=hybrid:NoSchedule
EOF

```


2. 설정 파일 검증

```
nodeadm config check --config-source file://$(pwd)/eksworkshop-nodeadm-config.yaml
```

3. AWS 환경 변수 설정

```
export AWS_DEFAULT_REGION=ap-northeast-2
```

4. 노드 초기화 실행

```
sudo -E nodeadm init --config-source file://$(pwd)/eksworkshop-nodeadm-config.yaml
```

SSM ACTIVATION CODE 발급 CLI - 최초 발급 시에만 확인되며 24시간 유지

```
aws ssm create-activation \
  --region <AWS_REGION> \
  --default-instance-name <인스턴스이름> \
  --description "Activation for EKS hybrid nodes" \
  --iam-role <Hybrid_Nodes_IAM_Role> \
  --registration-limit <최대노드수>
```

Private Harbor 연결

1. 요약

EKS Hybrid Node는 `EKS_HYBRID_ASSETS_URL` 환경변수로 artifact 다운로드 base URL을 오버라이드할 수 있습니다
즉,

```
https://hybrid-assets.eks.amazonaws.com/...
```

v

```
https://harbor.example.com/eks-hybrid-assets/...
```

로 1:1 구조 미러링을 해주면 됩니다.

2. 권장 아키텍처 (Best Practice)

```
EKS Hybrid Node
|
|  HTTPS (443)
v
Private Harbor (Project: eks-hybrid-assets)
|
|  (초기 동기화 시에만)
v
```

hybrid-assets.eks.amazonaws.com

- 운영 시에는 외부 인터넷 완전 차단 가능
- Harbor는 OCI image repo ❌
- Generic HTTP Artifact repo 역할로 사용

3. Harbor 쪽 구성 방법

(1) PROJECT 생성

Project Name: eks-hybrid-assets
Access Level: Private

(2) 디렉터리 구조 (중요)

Harbor는 내부적으로 blob 저장소지만,
외부에서는 "URL path" 그대로 접근 가능해야 합니다.

예시:

```
eks-hybrid-assets/  
├── artifacts/  
│   ├── 1.33.0/  
│   │   ├── 2025-05-01/  
│   │   │   ├── bin/  
│   │   │   │   ├── linux/  
│   │   │   │   │   ├── amd64/  
│   │   │   │   │   │   ├── kubelet  
│   │   │   │   │   │   ├── kubelet.sha256  
│   │   │   │   │   │   ├── kubectrl  
│   │   │   │   │   │   ├── kubectrl.sha256  
│   │   │   │   │   │   ├── cni-amd64-v0.6.0.tgz  
│   │   │   │   │   │   └── ...
```

- AWS URL path를 100% 동일하게 유지하는 것이 중요

(3) 아티팩트 미러링 스크립트 예시

```
BASE=https://hybrid-assets.eks.amazonaws.com/artifacts/1.33.0/2025-05-01  
TARGET=https://harbor.example.com/eks-hybrid-assets/artifacts/1.33.0/2025-05-01  
for f in \  
    bin/linux/amd64/kubelet \  
    bin/linux/amd64/kubelet.sha256 \  
    bin/linux/amd64/kubectrl \  
    bin/linux/amd64/kubectrl.sha256do  
do  
    curl -O $BASE/$f  
    curl -u admin:password -T $f $TARGET/$f  
done
```

- ✓ checksum 파일도 반드시 같이 업로드
- ✓ EKS hybrid bootstrap에서 sha256 검증 수행

4. EKS Hybrid Node 쪽 설정 (가장 중요)

- BOOTSTRAP 환경 변수

HYBRID NODE 설치 전에 다음을 반드시 EXPORT:

```
export EKS_HYBRID_ASSETS_URL=https://harbor.example.com/eks-hybrid-assets
```

또는 systemd drop-in:

```
# /etc/systemd/system/eks-hybrid-node.service.d/assets.conf
[Service]
Environment="EKS_HYBRID_ASSETS_URL=https://harbor.example.com/eks-hybrid-assets"
```

그러면 내부적으로:

```
${EKS_HYBRID_ASSETS_URL}/artifacts/1.33.0/...
```

로 다운로드 시도합니다.

5. 인증 방식 (Private Harbor인 경우)

옵션1. BASIC AUTH (가장 흔함)

```
export EKS_HYBRID_ASSETS_USERNAME=harbor-user export EKS_HYBRID_ASSETS_PASSWORD=harbor-
```

또는 .netrc

```
machine harbor.example.com
login harbor-user
password harbor-password
```

옵션2. MTLS (보안)

Harbor + Nginx 앞단에 Client cert
Hybrid node에 client cert 배포

6. harbor 구성 관련 가이드

- EKS Hybrid node 구성의 직접적인 구성 가이드는 없으나 위와 아래 링크 참고 가능
- harbor 구성 가이드 : <https://goharbor.io/docs/1.10/install-config/>

- EKS Anywhere harbor 구성 예시 : <https://anywhere.eks.amazonaws.com/docs/packages/harbor/addharbor/#install>

하이브리드 노드 업그레이드 및 패치

아래 작업은 NODE를 `cordon` 해서 트래픽 유입을 막고 `drain`을 통해서 노드를 분리하는 일반적인 node 연결 방법과 유사합니다.

참고 : https://docs.aws.amazon.com/ko_kr/eks/latest/userguide/hybrid-nodes-upgrade.html

클러스터의 하이브리드 노드 업그레이드

인플레이스 업그레이드 프로세스는 `nodeadm upgrade`를 사용하여 새로운 물리적 또는 가상 호스트 및 전환 마이그레이션 전략을 사용하지 않고 하이브리드 노드용 Kubernetes 버전을 업그레이드하는 것을 말합니다. 이 `nodeadm upgrade` 프로세스는 하이브리드 노드에서 실행되는 기존 구형 Kubernetes 구성 요소를 종료하고, 기존 구형 Kubernetes 구성 요소를 제거하고, 새 대상 Kubernetes 구성 요소를 설치하고, 새 대상 Kubernetes 구성 요소를 시작합니다. 하이브리드 노드에서 실행되는 애플리케이션에 미치는 영향을 최소화하기 위해 한 번에 하나의 노드를 업그레이드하는 것이 좋습니다. 이 프로세스의 지속 시간은 네트워크 대역폭 및 지연 시간에 따라 달라집니다.

다음 명령을 사용하여 업그레이드하는 노드를 예약 불가로 표시합니다. 이는 업그레이드하는 노드에서 새 포드가 예약되거나 다시 예약되지 않도록 하기 위한 것입니다. 자세한 내용은 Kubernetes 문서의 [kubectl cordon](#)을 참조하세요. `NODE_NAME`을 업그레이드하려는 하이브리드 노드의 이름으로 바꿉니다.

```
kubectl cordon NODE_NAME
```

다음 명령을 사용하여 업그레이드하려는 노드를 드레인합니다. 노드 드레인에 대한 자세한 내용은 Kubernetes 설명서의 [Safely Drain a Node](#)을 참조하세요. `NODE_NAME`을 업그레이드하려는 하이브리드 노드의 이름으로 바꿉니다.

```
kubectl drain NODE_NAME --ignore-daemonsets --delete-emptydir-data
```

업그레이드하려는 하이브리드 노드에서 `nodeadm upgrade`를 실행합니다. `root/sudo` 권한이 있는 사용자와 함께 `nodeadm`를 실행해야 합니다. 노드 이름은 AWS SSM 및 AWS IAM Roles Anywhere 자격 증명 공급자 모두에 대한 업그레이드를 통해 보존됩니다. 업그레이드 프로세스 중에는 자격 증명 공급자를 변경할 수 없습니다. [하이브리드 노드 nodeadm 참조](#)의 구성 값은 `nodeConfig.yaml` 섹션을 참조하세요. `K8S_VERSION`을 업그레이드할 대상 Kubernetes 버전으로 바꿉니다.

```
nodeadm upgrade K8S_VERSION -c file://nodeConfig.yaml
```

업그레이드한 후 노드에서 포드를 예약하도록 허용하려면 다음을 입력합니다. `NODE_NAME`을 노드의 이름으로 바꿉니다.

```
kubectl uncordon NODE_NAME
```

하이브리드 노드의 상태를 확인하고 노드가 종료될 때까지 기다렸다가 준비 상태로 새 Kubernetes 버전에서 다시 시작합니다.

```
kubectl get nodes -o wide -w
```

하이브리드 노드의 패치 보안 업데이트

- 참고 : https://docs.aws.amazon.com/ko_kr/eks/latest/userguide/hybrid-nodes-security.html

